



FRONT-END JS

Clase N° 8 | Git & Github

¡Les damos la bienvenida! 🚀

👤 Vamos a comenzar a grabar la clase.

Índice

1

8. Git y Github

- Git: Descarga de Git
- Crear un repositorio externo (Github)
- Comandos básicos (Init, commit, push)

2

9. Introducción a JavaScript

- ¿Qué es y para qué se usa JavaScript?
- Variable: ¿qué es y cómo declararla? Tipos
- Asignación y cambio del valor
- Operadores aritméticos
- Conversión a entero y flotante

Git y GitHub



¿Qué es Git?

Es un sistema de control de versiones Git permite guardar versiones de tu proyecto a lo largo del tiempo.

Te permite:



Viajar en el tiempo

Recupera versiones anteriores del proyecto si algo sale mal.



Trabajar en equipo

Facilita la colaboración sin conflictos entre desarrolladores.

¿Qué es GitHub?

GitHub es un servicio web que aloja repositorios Git. Utilizada comúnmente como una red social de desarrolladores y portafolio profesional.

Te permite:

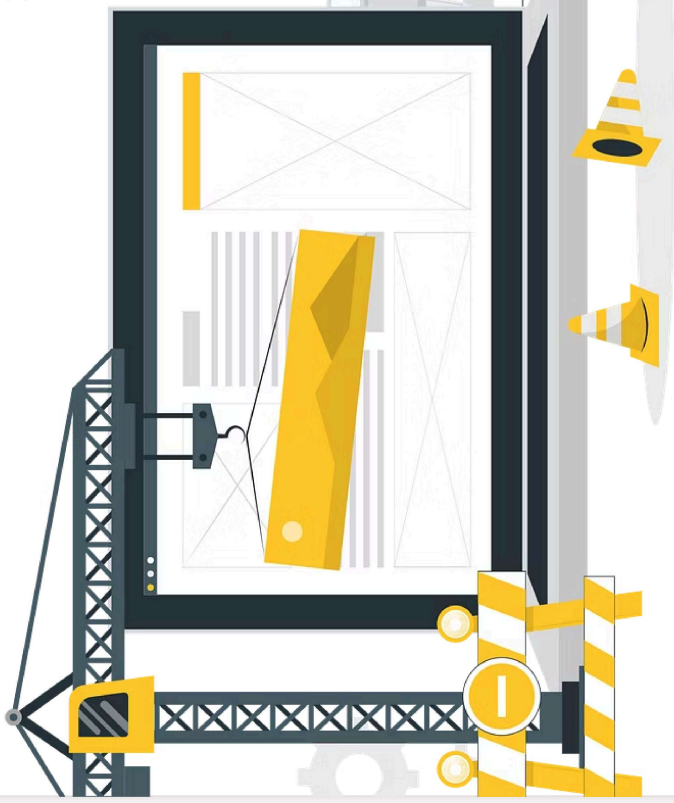
Colaboración global

Permite subir, compartir código y colaborar con desarrolladores de todo el mundo.

Funciones sociales

Podés colaborar en proyectos, subir cambios y ver el progreso que va teniendo tu proyecto a lo largo del tiempo





Instalación de Git

Windows

Descargá el instalador desde **Git SCM** y seguí las instrucciones.

Mac

Usa Homebrew y ejecuta `'brew install git'` en la terminal.

Linux

En distribuciones Debian/Ubuntu, usa:

`'sudo apt-get install git'`.

Configuración inicial de Git

Luego de instalar Git es necesario configurarlo. Deberás abrir la **terminal o consola** y hacer uso de los siguientes comandos:

 **git config --global user.name "Tu Nombre"**

git config --global user.email "tuemail@ejemplo.com"

Esto le dice a Git cuál es tu nombre y tu email para que puedas hacer *commits* (que son como "guardados" o "versiones").



```
emilianospinoso@ARJRFFGK3: ~/Escritorio
emilianospinoso@ARJRFFGK3: ~/Escritorio$ git config --global user.name "emiliano spinoso"
```

Conceptos Clave en Git



Repositorio (o "Repo")

Espacio donde se guarda todo el proyecto y su historial. Puede ser local (almacenado en tu computadora) o remoto (en GitHub).



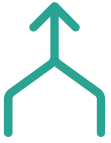
Branch (o "Rama")

Permite probar cambios en el código sin afectar lo que ya funciona. Si el resultado es satisfactorio, se puede unir a la rama principal (generalmente *main* o *master*).



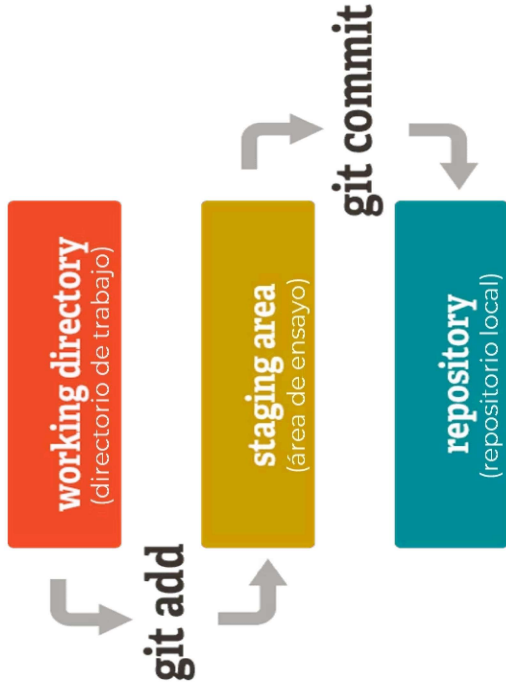
Commit

Guarda el estado actual del proyecto, registrando los cambios realizados como una "foto" en el tiempo



Merge (o "fusión")

Proceso de unir dos ramas. Después de probar algo nuevo en una rama, se puede unir al código principal mediante un `merge`



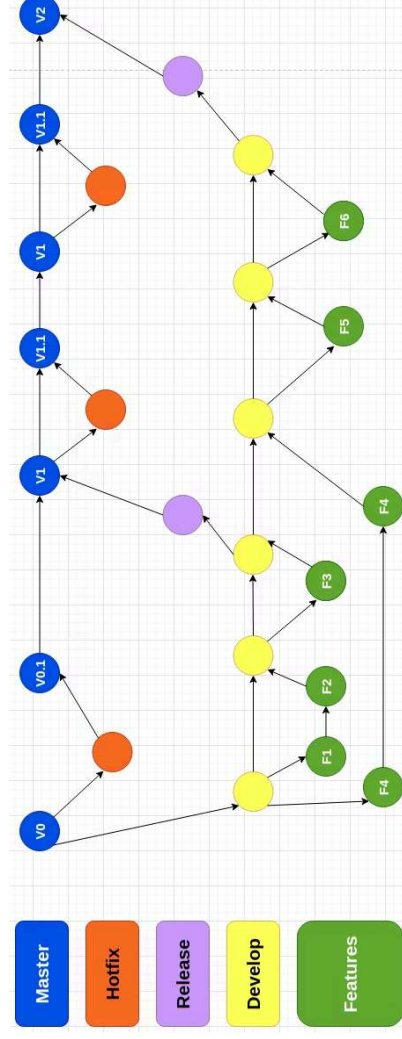
Comandos básicos de Git

- **git init:** Inicializa un nuevo repositorio en la carpeta actual. Esto crea un `.git`, que es la carpeta donde se guardan todos los cambios que vas a realizar. `git init`
- **git add:** Este comando selecciona los archivos que querés "guardar". Podés agregar archivos específicos o todo de una, con: `git add .`
- **git commit:** Crea un "snapshot" de los archivos agregados y guarda un mensaje descriptivo de los cambios. `git commit -m "Mensaje descriptivo de los cambios"`
- **git push:** Sube los commits de tu repositorio local a uno remoto en GitHub. `git push origin main`

Una rama (branch) es una línea paralela de desarrollo que te permite trabajar en nuevas funcionalidades, arreglos o pruebas sin modificar la rama principal. Podés fusionar (merge) los cambios de tu rama con la rama principal si todo funciona correctamente.



Son esenciales para un flujo de trabajo organizado, permiten desarrollar nuevas funcionalidades o corregir errores sin afectar la rama principal (*main* o *master*).



Crear y trabajar con ramas

Al trabajar en una nueva funcionalidad o corrección de errores, lo primero es crear una nueva rama. Esto permite aislar los cambios y garantizar que no afecten la rama principal hasta confirmar que todo funciona correctamente.

Comandos para crear una nueva rama:

```
git checkout -b nombre-de-la-rama
```

```
git checkout -b crea una nueva rama y cambia automáticamente a esa rama.
```

Reemplazá `nombre-de-la-rama` con un nombre descriptivo para la rama, como

```
feature-nueva-funcionalidad o bugfix-corregir-bug.
```

Ejemplo:

```
git checkout -b feature-agregar-formulario-contacto
```

Ahora estás trabajando en una nueva rama llamada `feature-agregar-formulario-contacto`.

Hacer Cambios en la Nueva Rama

Podés realizar todos los cambios necesarios en una rama sin afectar el resto del proyecto. Modifica el código, agregá archivos, y asegurate de hacer commits regularmente para registrar tus cambios.

Ejemplo:

```
git add .
```

```
git commit -m "Agregado nuevo formulario de contacto"
```

Subir la Rama al Repositorio Remoto (GitHub):

Una vez que estás listo para compartir tu rama o realizar una revisión del código, podés subirla a GitHub.

Comando:

```
git push origin nombre-de-la-rama
```

Ejemplo:

```
git push origin feature-agregar-formulario-contacto
```

Fusionar ramas (Merge)

1

Cambiar a rama principal: Usa 'git checkout main' para volver a la rama principal.

2

Realizar el merge: Ejecuta 'git merge nombre-rama' para fusionar los cambios.

- **Ejemplo:** git merge feature-agregar-formulario-contacto
- Combina los cambios de la rama feature-agregar-formulario-contacto con la rama principal.

3

Eliminar la Rama (Opcional): Una vez que la rama ha sido fusionada y no se usa más, podés eliminarla

- Comando: `git branch -d nombre-de-la-rama`

- Ejemplo: `git branch -d feature-agregar-formulario-contacto`

4 Resolver Conflictos:

Si durante un merge Git detecta conflictos entre tu rama y la principal, deberás resolverlos manualmente. Git señalará en el archivo afectado dónde están los conflictos, y tendrás que decidir qué cambios conservar.

Flujo de trabajo con Git y GitHub

1 Inicio de una nueva tarea:

```
git checkout -b feature-nueva-funcionalidad
```

2

Realizar cambios y registrarlos en la rama mediante commits:

```
git add .
```

3

Subida de cambios:

```
git push origin feature-nueva-funcionalidad.
```

```
git commit -m "Agregada nueva funcionalidad"
```

4

Finalización: Fusiónás los cambios

```
git checkout main
```

Eliminás la rama si ya no es necesaria:

```
git branch -d feature-nueva-funcionalidad
```

5

```
git merge feature-nueva-funcionalidad
```

Crear un repositorio



Crear nuevo repositorio

Clic en el botón "New" en la página principal. Elegí un nombre descriptivo



Vincular repositorio local con GitHub

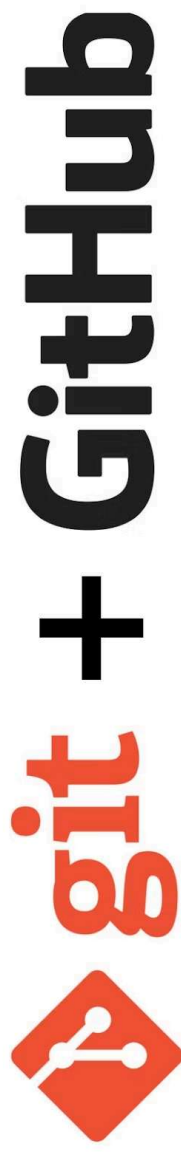
```
git remote add origin https://github.com/tu-usuario/proyecto.git
```



Subir cambios a GitHub

Después de hacer cambios y commits, podés subirlos:

```
git push -u origin main
```



Resolución de conflictos en Git

Los conflictos surgen cuando dos personas, o dos ramas diferentes, modifican las mismas líneas de código. Git no puede decidir cuál versión conservar, solicita resolver los conflictos manualmente.

1 Identificar el conflicto:

```
git status
```

Los archivos en conflicto **aparecerán en rojo**.

2 Abrir el archivo en conflicto

Accedé al archivo en conflicto por medio de tu editor de código preferido.

3 Resolver el conflicto:

Elegir qué código mantener. Optar por una de las versiones o hacer una mezcla de ambas.

4 Agregar el archivo resuelto:

Volvés a agregar el archivo al *staging area*:

```
git add nombre-del-archivo
```

5 Hacer un commit:

Hacer un commit que indique que resolviste el conflicto:

```
git commit -m "Conflicto resuelto"
```

6 Continuar:

Luego de resolver el conflicto y hacer *commit*, continuar con el flujo de trabajo normal (push, merge, etc.).

GitHub Desktop

Interfaz gráfica

Facilita la gestión de repositorios sin usar la terminal.

Visualización de cambios

Muestra diferencias entre versiones de manera visual.

Operaciones simplificadas

Realiza commits, push y pull con pocos clics.



GitHub Desktop ¿Cómo usarlo?

1

Descarga e instalación

Descargá GitHub Desktop desde desktop.github.com e instalalo en tu computadora.

2

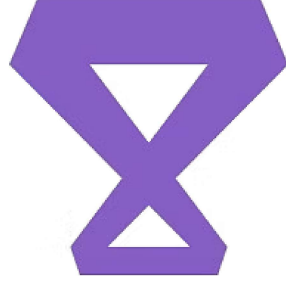
Clonar repositorio

Podés clonar un repositorio existente o crear uno nuevo directamente desde la app.

3

Commits y Push

Cada vez que modifiques algo en tu proyecto, podés hacer commits y enviar esos cambios a GitHub sin necesidad de abrir la terminal.



Visual Studio Code y Git

Integración nativa

VS Code tiene soporte incorporado para Git.

Control de fuente

Gestiona cambios desde la barra lateral de Source Control.

Resolución de conflictos

Interfaz visual para resolver conflictos de merge.

Terminal integrada

Ejecuta comandos Git directamente desde VS Code.

Visual Studio Code ¿Cómo usarlo?

1 Abrió tu proyecto

Iniciá Visual Studio Code y abrí el proyecto que querés gestionar con Git

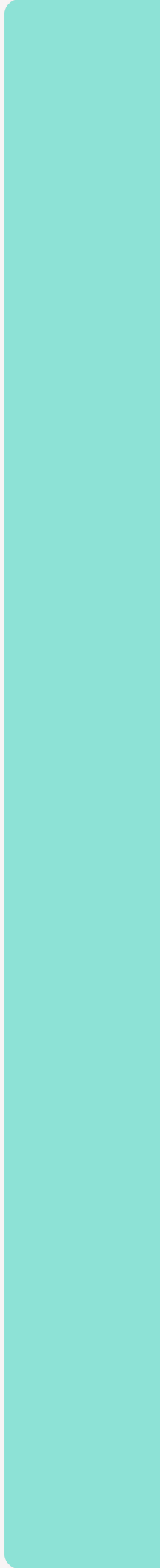
2 Usá la barra lateral de Git

Hacé clic en el ícono de Source Control en la barra lateral para ver los cambios y hacer commits

3 Sincronizá con GitHub

Conectá tu cuenta de GitHub para gestionar repositorios remotos directamente desde VS Code

Ejercicios Prácticos





Ejercicio Práctico #1

Optativo | No entregable

Crear una nueva rama en Git

Crear una nueva rama en tu repositorio Git además de la rama principal (main o master). Nombrá la nueva rama de acuerdo con la funcionalidad o característica que estás desarrollando. Asegurate de hacer un commit en la nueva rama y luego subila al repositorio remoto en GitHub.

Tips:

- Creación de la rama: Usá el comando `git checkout -b nombre-de-la-rama` para crear y cambiarte a la nueva rama al mismo tiempo.
- Verificar la rama actual: Ejecutá `git branch` para verificar que estés en la rama correcta antes de hacer cambios.
- Hacer commits en la nueva rama: No olvides usar `git add` , seguido de `git commit -m "Descripción de los cambios"` para registrar los cambios en la nueva rama.
- Subir la nueva rama a GitHub: Usá `git push origin nombre-de-la-rama` para subir tu nueva rama al repositorio remoto.



Ejercicio Práctico #2

Optativo | No entregable

Repositorio en Github

Crear un repositorio en GitHub para tu proyecto, subir el código actual y realizar commits documentando los cambios.

Asegurarse de que el repositorio esté configurado para futuras actualizaciones.

A continuación te mostramos cómo nombrar el repositorio:



Nombre del repositorio: proyecto-final-ecommerce-[nombre-apellido]

► **Tips:**

Pre-Entrega de Proyecto

Revisión de progreso



Estamos acercándonos a un **punto clave en el desarrollo del proyecto**, donde tendrán la oportunidad de mostrar lo que han avanzado hasta ahora, **compartir su progreso y recibir comentarios** que les ayudarán a pulir detalles antes de la entrega final.

Entregable

1 Recopilación

Reunir todo el trabajo desarrollado hasta la clase 8.

2 Consolidación

Oportunidad para consolidar ideas y asegurar el rumbo correcto.

3 Feedback

Recibirán comentarios para fortalecer el proyecto.



Pre-Entrega de Proyecto

Obligatorio | Entregable

A partir de la **Clase N° 8** tendrás **7 días** de corrido para realizar la autoevaluación y la entrega en el campus virtual

Cuestionario de Autoevaluación

Pre-Entrega de Proyecto:

- Te permitirá reflexionar sobre tu propio aprendizaje, progreso y cumplimiento de las consignas o rúbricas previamente establecidas y en caso de ser necesario realizar las modificaciones o ajustes correspondientes antes de realizar la Pre-Entrega.
- Se encontrará disponible en la Ruta N°2 de Campus Virtual.

- Se evaluará la aplicación de los conocimientos adquiridos en la realización de un proyecto.
- La realización progresiva de los "**Ejercicios prácticos**" y el seguimiento de las rúbricas de la "**Ruta de Avance**" los guiará paso a paso hacia la realización de la "**Pre - Entrega**" y el "**Proyecto Integrador Final**".
- Se entregará en la Ruta N°2 de Campus Virtual.

Pre-Entrega de Proyecto

Obligatorio | Entregable

A partir de la **Clase N° 8** tendrás **7 días** de corrido para realizar la autoevaluación y la entrega en el campus virtual

1. Estructura Básica de HTML.

- **Estructura semántica:** El HTML debe estar dividido en las etiquetas semánticas principales: **header**, **nav**, **main**, **section**, **footer**.

README.md: Incluir un archivo que explique brevemente el propósito de la página.

2. Formulario de Contacto.

Formulario funcional: Crear un formulario de contacto con campos para **nombre**, **correo electrónico** y **mensaje**, utilizando **Formspree** para manejar el envío de datos.



Pre-Entrega de Proyecto

Obligatorio | Entregable

3. Estilos Básicos Aplicados con CSS

- **Archivo styles.css:** El proyecto debe contar con un archivo CSS externo que incluya:
 - Estilos básicos aplicados a las secciones de **header**, **footer** y **lista de navegación**.
 - **Fuentes de Google Fonts** correctamente implementadas.

Propiedades de **background** aplicadas en alguna sección de la página (color, imagen, degradado, etc.).

4. Diseño Responsivo con Flexbox y Grid

- **Sección "Productos":** Organizada en **cards** de forma responsiva utilizando **Flexbox**.
- **Sección "Reseñas":** Organizada utilizando **Grid**, con una distribución lógica y estética.
- **Sección "Contacto":** Debe ser responsiva mediante el uso de **Media Queries** para adaptarse a diferentes tamaños de pantalla.



Pre-Entrega de Proyecto

Obligatorio | Entregable

5. Contenido Multimedia y Navegación

- **Multimedia:** deberá incluir archivos multimedia (imagenes, video o iframe) correctamente integrado en la página.

Lista de navegación: Implementar una **lista desordenada** con enlaces que simulen una navegación interna (Inicio, Productos, Contacto, etc.).

6. Subida del Proyecto.

El proyecto debe estar subido a un **hosting gratuito** (Netlify o GitHub Pages), con una **URL funcional** para visualizar el sitio.

Formato de Entrega

1

Carpeta en Drive

Crear una carpeta en drive (pública) que contenga los archivos y carpetas que conforman tu proyecto.

2

Link de Entrega

Compartir el link en el apartado de entrega en el Campus Virtual

3

Hosting

Compartir el link de Github Pages o subida a servidor.

Resumen: Pre Entrega de Proyecto

1

Cuestionario de Autoevaluación

Completar el cuestionario de reflexión.

2

Requisitos para la entrega

- Estructura Básica de HTML
- Formulario de Contacto.
- Estilos Básicos con CSS y Diseño Responsivo
- Contenido Multimedia y Navegación

3

Pre-Entrega del Proyecto

Subir proyecto y compartir enlaces.